

SLIP 1

Q.1) Create an HTML form for Login and write a JavaScript to validate email ID and Password using Regular Expression.

```
<!DOCTYPE html>
<html>
<head>
  <title>Login Form</title>
  <script>
    function validateForm() {
      let email = document.getElementById("email").value;
      let password = document.getElementById("password").value;
      let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      let passwordPattern = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/;

      if (!emailPattern.test(email)) {
        alert("Invalid Email ID");
        return false;
      }

      if (!passwordPattern.test(password)) {
        alert("Password must be at least 8 characters long and include:
        \n- 1 uppercase\n- 1 lowercase\n- 1 number\n- 1 special character");
        return false;
      }

      alert("Login Successful!");
      return true;
    }
  </script>
</head>
<body>

  <h2>Login Form</h2>

  <form onsubmit="return validateForm()">
    <label>Email:</label><br>
    <input type="text" id="email" required><br><br>

    <label>Password:</label><br>
    <input type="password" id="password" required><br><br>

    <input type="submit" value="Login">
  </form>

</body>
</html>
```

Q.2) Create an HTML form that contain the Student Registration details and write a JavaScript to validate Student first and last name as it should not contain other than alphabets and age should be between 18 to 50.

```

<!DOCTYPE html>
<html>
<head>
  <title>Student Registration Form</title>
  <script>
    function validateForm() {
      let fname = document.getElementById("fname").value;
      let lname = document.getElementById("lname").value;
      let age = document.getElementById("age").value;

      let namePattern = /^[A-Za-z]+$\/;

      if (!namePattern.test(fname)) {
        alert("First Name should contain only alphabets.");
        return false;
      }

      if (!namePattern.test(lname)) {
        alert("Last Name should contain only alphabets.");
        return false;
      }

      if (age < 18 || age > 50) {
        alert("Age must be between 18 and 50.");
        return false;
      }

      alert("Registration Successful!");
      return true;
    }
  </script>
</head>
<body>

  <h2>Student Registration Form</h2>

  <form onsubmit="return validateForm()">
    <label>First Name:</label><br>
    <input type="text" id="fname" required><br><br>

    <label>Last Name:</label><br>
    <input type="text" id="lname" required><br><br>

    <label>Age:</label><br>
    <input type="number" id="age" required><br><br>

    <input type="submit" value="Register">
  </form>

</body>
</html>

```

SLIP 2

Q.1) Create a Node.js file that will convert the output "Full Stack!" into reverse string.

```

let str = "Full Stack!";

let reversed = str.split("").reverse().join("");

console.log("Original String:", str);
console.log("Reversed String:", reversed);

```

Q.2) Using node js create a web page to read two file names from user and append contents of first file into second file.

```

<!DOCTYPE html>
<html>
<head>
  <title>File Append</title>
</head>
<body>

<h2>Append File Content</h2>

<form action="/append" method="post">
  <label>First File Name:</label><br>
  <input type="text" name="file1" required><br><br>

  <label>Second File Name:</label><br>
  <input type="text" name="file2" required><br><br>

  <input type="submit" value="Append">
</form>

</body>
</html>

```

__First install required modules:

```

npm init -y
npm install express body-parser

```

__Server Code:

```

const express = require("express");
const fs = require("fs");
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});

app.post("/append", (req, res) => {
  const file1 = req.body.file1;
  const file2 = req.body.file2;

  fs.readFile(file1, "utf8", (err, data) => {
    if (err) {
      return res.send("Error reading first file: " + err.message);
    }
  })

```

```

        fs.appendFile(file2, data, (err) => {
            if (err) {
                return res.send("Error appending to second file: " +
err.message);
            }

            res.send("Content successfully appended!");
        });
    });
});

app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});

```

SLIP 3

Q.1) Using node js create a User Login System.

```

<!DOCTYPE html>
<html>
<head>
    <title>User Login</title>
</head>
<body>

<h2>Login Form</h2>

<form action="/login" method="post">
    <label>Username:</label><br>
    <input type="text" name="username" required><br><br>

    <label>Password:</label><br>
    <input type="password" name="password" required><br><br>

    <input type="submit" value="Login">
</form>

</body>
</html>

```

___. Node.js File (app.js)

```

const express = require("express");
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

const validUser = {
    username: "admin",
    password: "admin123"
};

app.get("/", (req, res) => {
    res.sendFile(__dirname + "/index.html");
});

```

```

app.post("/login", (req, res) => {
  const username = req.body.username;
  const password = req.body.password;

  if (username === validUser.username && password === validUser.password) {
    res.send("<h2>Login Successful &#xA0;&#xA0;&#xA0;</h2>");
  } else {
    res.send("<h2>Invalid Username or Password &#xA0;&#xA0;</h2>");
  }
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});

```

__Steps to Run

```

npm init -y
npm install express body-parser
node app.js
Open browser at http://localhost:3000

```

Q.2) Create a node.js file that Select all records from the "Teacher" table, and find the Teachers whose salary is greater than 20,000.

```

const mysql = require("mysql");

const con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "your_database_name"
});

con.connect(function(err) {
  if (err) throw err;

  con.query("SELECT * FROM Teacher", function(err, result) {
    if (err) throw err;
    console.log("All Records:");
    console.log(result);
  });

  con.query("SELECT * FROM Teacher WHERE salary > 20000", function(err, result) {
    if (err) throw err;
    console.log("Teachers with salary > 20000:");
    console.log(result);
  });
});

```

__To Run

```

npm install mysql
node app.js

```

Q.1) Using node js create an eLearning System.

```
<!DOCTYPE html>
<html>
<head>
  <title>eLearning System</title>
</head>
<body>

<h2>Available Courses</h2>

<form action="/enroll" method="post">
  <label>Name:</label><br>
  <input type="text" name="name" required><br><br>

  <label>Select Course:</label><br>
  <select name="course">
    <option value="Full Stack Development">Full Stack Development</option>
    <option value="Data Science">Data Science</option>
    <option value="Machine Learning">Machine Learning</option>
  </select><br><br>

  <input type="submit" value="Enroll">
</form>

</body>
</html>
```

__ Node.js File (app.js)

```
const express = require("express");
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});

app.post("/enroll", (req, res) => {
  const name = req.body.name;
  const course = req.body.course;

  res.send("<h2>" + name + " enrolled in " + course + " successfully!</h2>");
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

__Steps to Run

```
npm init -y
npm install express body-parser
node app.js
```

Q.2) Create an HTML form using AngularJS that contain the Student Registration details
and validate Student first and last name as it should not contain other than alphabets

and age should be between 18 to 50 and display greeting message depending on current time using ng-show (e.g. Good Morning, Good Afternoon, etc.)(Use AJAX).

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Student Registration</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>Student Registration Form</h2>

<form name="regForm" ng-submit="submitForm()">

  First Name:<br>
  <input type="text" ng-model="fname" required>
  <span ng-show="regForm.$submitted && !fname.match(namePattern)">
    Only alphabets allowed
  </span><br><br>

  Last Name:<br>
  <input type="text" ng-model="lname" required>
  <span ng-show="regForm.$submitted && !lname.match(namePattern)">
    Only alphabets allowed
  </span><br><br>

  Age:<br>
  <input type="number" ng-model="age" required>
  <span ng-show="regForm.$submitted && (age < 18 || age > 50)">
    Age must be between 18 and 50
  </span><br><br>

  <input type="submit" value="Register">
</form>

<br>

<h3 ng-show="greeting">{{greeting}}, {{fname}}!</h3>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope, $http) {

  $scope.namePattern = /^[A-Za-z]+$/;

  $scope.submitForm = function() {

    if (!$scope.namePattern.test($scope.fname) ||
        !$scope.namePattern.test($scope.lname) ||
        $scope.age < 18 || $scope.age > 50) {
      return;
    }

    let hour = new Date().getHours();

    if (hour < 12)
      $scope.greeting = "Good Morning";
    else if (hour < 17)
```

```

        $scope.greeting = "Good Afternoon";
    else
        $scope.greeting = "Good Evening";

    $http({
        method: "GET",
        url: "https://jsonplaceholder.typicode.com/posts/1"
    }).then(function(response) {
        console.log("AJAX call success");
    }, function(error) {
        console.log("AJAX call failed");
    });
};

});
</script>

</body>
</html>

```

SLIP 5

Q.1) Create a Node.js file that writes an HTML form, with an upload field.

__Node.js

```

const http = require("http");
const fs = require("fs");
const formidable = require("formidable");

http.createServer(function (req, res) {
    if (req.url == "/upload") {
        const form = new formidable.IncomingForm();

        form.parse(req, function (err, fields, files) {
            const oldPath = files.file.filepath;
            const newPath = __dirname + "/" + files.file.originalFilename;

            fs.rename(oldPath, newPath, function (err) {
                if (err) throw err;
                res.write("File uploaded successfully!");
                res.end();
            });
        });
    } else {
        res.writeHead(200, { "Content-Type": "text/html" });
        res.write(`
            <html>
            <body>
                <h2>File Upload Form</h2>
                <form action="/upload" method="post" enctype="multipart/form-
data">
                    <input type="file" name="file"><br><br>
                    <input type="submit" value="Upload">
                </form>
            </body>
            </html>
        `);
        res.end();
    }
});

```



```
    }  
  }).listen(3000);
```

___ Run Steps
npm init -y
npm install formidable
node app.js

Open browser at http://localhost:3000

Q.2) Using angular js create a SPA to carry out validation for a username entered in a textbox. If the textbox is blank, alert "Enter username". If the number of characters is less than three, alert "Username is too short". If value entered is appropriate the print "Valid username" and password should be minimum 8 characters.

```
<!DOCTYPE html>  
<html ng-app="myApp">  
<head>  
  <title>Username Validation SPA</title>  
  <script  
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></  
script>  
</head>  
<body ng-controller="myController">  
  
<h2>User Validation Form</h2>  
  
<form ng-submit="validate()">  
  
  Username:<br>  
  <input type="text" ng-model="username"><br><br>  
  
  Password:<br>  
  <input type="password" ng-model="password"><br><br>  
  
  <input type="submit" value="Submit">  
</form>  
  
<h3>{{message}}</h3>  
  
<script>  
var app = angular.module("myApp", []);  
  
app.controller("myController", function($scope) {  
  
  $scope.validate = function() {  
  
    if (!$scope.username || $scope.username.trim() === "") {  
      alert("Enter username");  
      return;  
    }  
  
    if ($scope.username.length < 3) {  
      alert("Username is too short");  
      return;  
    }  
  
    if (!$scope.password || $scope.password.length < 8) {  
      alert("Password should be minimum 8 characters");  
    }  
  }  
}
```

```

        return;
    }

    $scope.message = "Valid username";
};

});
</script>

</body>
</html>

```

SLIP 6

Q.1) Write angular JS by using ng-click directive to display an alert message after clicking the element.

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
    <title>ng-click Example</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>ng-click Demo</h2>

<button ng-click="showAlert()">Click Me</button>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope) {
    $scope.showAlert = function() {
        alert("Button clicked successfully!");
    };
});
</script>

</body>
</html>

```

Q.2) Create a Node.js file that opens the requested file and returns the content to the client. If anything goes wrong, throw a 404 error.

```

const http = require("http");
const fs = require("fs");
const url = require("url");

http.createServer(function (req, res) {
    const q = url.parse(req.url, true);
    const filename = "." + q.pathname;

    fs.readFile(filename, function(err, data) {
        if (err) {
            res.writeHead(404, {"Content-Type": "text/html"});

```

```

        res.write("404 Not Found");
        return res.end();
    }
    res.writeHead(200, {"Content-Type": "text/html"});
    res.write(data);
    return res.end();
});
}).listen(3000);

```

__Run
 node app.js
 http://localhost:3000/index.html

SLIP 7

Q.1) Create angular JS Application that show the current Date and Time of the System (Use Interval Service)

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Date & Time</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>Current Date and Time</h2>

<h3>{{currentDateTime | date:'yyyy-MM-dd HH:mm:ss'}}</h3>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope, $interval) {

  $scope.currentDateTime = new Date();

  $interval(function() {
    $scope.currentDateTime = new Date();
  }, 1000);

});
</script>

</body>
</html>

```

Q.2) Create a node js file named main.js for event-driven application. There should be a main loop that listens for events, and then triggers a callback function when one of those events is detected.

__main.js
 const EventEmitter = require("events");

```

const EventEmitter = new EventEmitter();

eventEmitter.on("greet", function(name) {
  console.log("Hello " + name);
});

eventEmitter.on("bye", function() {
  console.log("Goodbye!");
});

console.log("Waiting for events...");

setTimeout(function() {
  eventEmitter.emit("greet", "Student");
}, 2000);

setTimeout(function() {
  eventEmitter.emit("bye");
}, 4000);

__Run
node main.js

```

SLIP 8

Q.1) Create a Simple Web Server using node js

```

__ (app.js)

const http = require("http");

const server = http.createServer(function(req, res) {
  res.writeHead(200, { "Content-Type": "text/html" });
  res.write("<h2>Welcome to Simple Node.js Web Server</h2>");
  res.end();
});

server.listen(3000);

__Run
node app.js

__Open browser:
http://localhost:3000

```

Q.2) Using angular js display the 10 student details in Table format
(using ng-repeat directive use Array to store data)

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Student Table</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

```

```

<h2>Student Details</h2>

<table border="1" cellpadding="10">
  <tr>
    <th>ID</th>
    <th>Name</th>
    <th>Age</th>
  </tr>

  <tr ng-repeat="student in students">
    <td>{{student.id}}</td>
    <td>{{student.name}}</td>
    <td>{{student.age}}</td>
  </tr>
</table>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope) {
  $scope.students = [
    {id: 1, name: "Amit", age: 20},
    {id: 2, name: "Rahul", age: 21},
    {id: 3, name: "Sneha", age: 19},
    {id: 4, name: "Priya", age: 22},
    {id: 5, name: "Kiran", age: 20},
    {id: 6, name: "Neha", age: 21},
    {id: 7, name: "Rohit", age: 23},
    {id: 8, name: "Pooja", age: 20},
    {id: 9, name: "Vikas", age: 22},
    {id: 10, name: "Anjali", age: 19}
  ];
});
</script>

</body>
</html>

```

SLIP 9

Q.1) Create a Node.js file that writes an HTML form, with a concatenate two string.

___ Node.js File (app.js)

```

const http = require("http");
const querystring = require("querystring");

http.createServer(function(req, res) {
  if (req.method === "POST") {
    let body = "";

    req.on("data", function(chunk) {
      body += chunk;
    });
  }
});

```

```

    req.on("end", function() {
        const data = querystring.parse(body);
        const result = data.str1 + data.str2;

        res.writeHead(200, {"Content-Type": "text/html"});
        res.write("<h2>Concatenated String: " + result + "</h2>");
        res.end();
    });

} else {
    res.writeHead(200, {"Content-Type": "text/html"});
    res.write(`
        <html>
        <body>
            <h2>Concatenate Two Strings</h2>
            <form method="post">
                Enter String 1: <input type="text" name="str1"><br><br>
                Enter String 2: <input type="text" name="str2"><br><br>
                <input type="submit" value="Concatenate">
            </form>
        </body>
        </html>
    `);
    res.end();
}

}).listen(3000);

```

__Run
node app.js

__Open:
http://localhost:3000

Q.2) Create a Node.js file that opens the requested file and returns the content to the client
If anything goes wrong, throw a 404 error

__Node.js File (app.js)

```

const http = require("http");
const fs = require("fs");
const url = require("url");

http.createServer(function(req, res) {
    const q = url.parse(req.url, true);
    const filename = "." + q.pathname;

    fs.readFile(filename, function(err, data) {
        if (err) {
            res.writeHead(404, {"Content-Type": "text/html"});
            res.end("404 Not Found");
        } else {
            res.writeHead(200, {"Content-Type": "text/html"});
            res.end(data);
        }
    });
}).listen(3000);

```

__Run

node app.js

Open in browser:

http://localhost:3000/index.html

SLIP 10

Q.1) Create a Node.js file that demonstrate create college database and table in MySQL

__Node.js File (app.js)

```
const mysql = require("mysql");

const con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: ""
});

con.connect(function(err) {
  if (err) throw err;

  con.query("CREATE DATABASE college", function(err, result) {
    if (err) throw err;
    console.log("Database created");
  });

  const con2 = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "",
    database: "college"
  });

  con2.connect(function(err) {
    if (err) throw err;

    const sql = "CREATE TABLE student (id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(50), age INT)";
    con2.query(sql, function(err, result) {
      if (err) throw err;
      console.log("Table created");
    });
  });
});
```

__Run

npm install mysql

node app.js

Q.2) Write node js script to build Your Own Node.js Module. Use require (â™™httpâ™™)

module is a built-in Node module that invokes the functionality of the HTTP library to create a local server. Also use the export statement to make functions in

your module available externally. Create a new text file to contain the functions in your module called, `modules.js` and add this function to return today's date and time.

`__modules.js`

```
exports.getDateTime = function () {  
    return new Date();  
};
```

`__app.js`

```
const http = require("http");  
const myModule = require("./modules");  
  
http.createServer(function (req, res) {  
    res.writeHead(200, { "Content-Type": "text/html" });  
    res.write("Current Date and Time: " + myModule.getDateTime());  
    res.end();  
}).listen(3000);
```

`__Run`

`node app.js`

Open:

`http://localhost:3000`

SLIP 11

Q.1) Create a Node.js file that demonstrates create Movie database and table in MySQL.

`__app.js`

```
const mysql = require("mysql");  
  
const con = mysql.createConnection({  
    host: "localhost",  
    user: "root",  
    password: ""  
});  
  
con.connect(function(err) {  
    if (err) throw err;  
  
    con.query("CREATE DATABASE movie", function(err, result) {  
        if (err) throw err;  
        console.log("Database created");  
    });  
  
    const con2 = mysql.createConnection({  
        host: "localhost",  
        user: "root",  
        password: "",  
        database: "movie"  
    });  
});
```



```

    con2.connect(function(err) {
        if (err) throw err;

        const sql = "CREATE TABLE movies (id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100), year INT)";
        con2.query(sql, function(err, result) {
            if (err) throw err;
            console.log("Table created");
        });
    });
});

__Run
npm install mysql
node app.js

```

Q.2) Write node js application that transfer a file as an attachment on web and enables browser to prompt the user to download file using express js

```

__app.js

const express = require("express");
const path = require("path");

const app = express();

app.get("/download", function(req, res) {
    const file = path.join(__dirname, "sample.txt");
    res.download(file);
});

app.listen(3000);

__Run
npm install express
node app.js

Open:
http://localhost:3000/download

```

SLIP 12

Q.1) Create a node.js file that Select all records from the "customers" table, and display the result object on console.

```

__app.js

const mysql = require("mysql");

const con = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "",

```

```

        database: "your_database_name"
    });

    con.connect(function(err) {
        if (err) throw err;

        con.query("SELECT * FROM customers", function(err, result) {
            if (err) throw err;
            console.log(result);
        });
    });
});

```

__Run
 npm install mysql
 node app.js

Q.2) Create an HTML form for Student Feedback Form with Name, Email ID, Mobile No., feedback (Not good, good, very good, excellent) and write a JavaScript to validate all field using Regular Expression.

```

<!DOCTYPE html>
<html>
<head>
  <title>Student Feedback Form</title>
  <script>
    function validateForm() {
      let name = document.getElementById("name").value.trim();
      let email = document.getElementById("email").value.trim();
      let mobile = document.getElementById("mobile").value.trim();
      let feedback =
document.querySelector('input[name="feedback"]:checked');

      let namePattern = /^[A-Za-z ]+$/;
      let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      let mobilePattern = /^[0-9]{10}$/;

      if (!namePattern.test(name)) {
        alert("Enter valid name (alphabets only)");
        return false;
      }

      if (!emailPattern.test(email)) {
        alert("Enter valid email");
        return false;
      }

      if (!mobilePattern.test(mobile)) {
        alert("Enter valid 10-digit mobile number");
        return false;
      }

      if (!feedback) {
        alert("Please select feedback");
        return false;
      }

      alert("Feedback submitted successfully!");
      return true;
    }
  </script>
</head>
</html>

```

```

    </script>
</head>
<body>

<h2>Student Feedback Form</h2>

<form onsubmit="return validateForm()">

    Name:<br>
    <input type="text" id="name" required><br><br>

    Email ID:<br>
    <input type="text" id="email" required><br><br>

    Mobile No:<br>
    <input type="text" id="mobile" required><br><br>

    Feedback:<br>
    <input type="radio" name="feedback" value="Not good"> Not good<br>
    <input type="radio" name="feedback" value="Good"> Good<br>
    <input type="radio" name="feedback" value="Very good"> Very good<br>
    <input type="radio" name="feedback" value="Excellent"> Excellent<br><br>

    <input type="submit" value="Submit">

</form>

</body>
</html>

```

SLIP 13

Q.1) Create a Node.js file that will convert the output "HELLO WORLD!" into lower-case letters.

__app.js

```

let str = "HELLO WORLD!";

let lower = str.toLowerCase();

console.log("Original String:", str);
console.log("Lowercase String:", lower);

```

__Run
node app.js

Q.2) Create an HTML form that contain the Student Registration details and write a JavaScript to validate Student first and last name as it should not contain other than alphabets and age should be between 18 to 50.

```

<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
    <script>

```

```

function validateForm() {
    let fname = document.getElementById("fname").value.trim();
    let lname = document.getElementById("lname").value.trim();
    let age = parseInt(document.getElementById("age").value);

    let namePattern = /^[A-Za-z]+$/;

    if (!namePattern.test(fname)) {
        alert("First Name should contain only alphabets");
        return false;
    }

    if (!namePattern.test(lname)) {
        alert("Last Name should contain only alphabets");
        return false;
    }

    if (age < 18 || age > 50) {
        alert("Age must be between 18 and 50");
        return false;
    }

    alert("Registration Successful!");
    return true;
}
</script>
</head>
<body>

<h2>Student Registration Form</h2>

<form onsubmit="return validateForm()">

    First Name:<br>
    <input type="text" id="fname" required><br><br>

    Last Name:<br>
    <input type="text" id="lname" required><br><br>

    Age:<br>
    <input type="number" id="age" required><br><br>

    <input type="submit" value="Register">

</form>

</body>
</html>

```

SLIP 14

Q.1) Create a Simple Web Server using node js.

__app.js

```
const http = require("http");
```

```
http.createServer(function(req, res) {
  res.writeHead(200, { "Content-Type": "text/html" });
  res.write("<h2>Simple Node.js Web Server Running</h2>");
  res.end();
}).listen(3000);
```

__Run
node app.js

Open in browser:
http://localhost:3000

Q.2) Create an HTML form that contain the Employee Registration details and write a JavaScript to validate DOB, Joining Date, and Salary.

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Registration</title>
  <script>
    function validateForm() {
      let dob = document.getElementById("dob").value;
      let join = document.getElementById("join").value;
      let salary = document.getElementById("salary").value;

      let dobDate = new Date(dob);
      let joinDate = new Date(join);
      let today = new Date();

      if (dobDate >= today) {
        alert("DOB must be in the past");
        return false;
      }

      if (joinDate <= dobDate) {
        alert("Joining Date must be after DOB");
        return false;
      }

      if (joinDate > today) {
        alert("Joining Date cannot be in future");
        return false;
      }

      let salaryPattern = /^[0-9]+$;/;

      if (!salaryPattern.test(salary) || salary <= 0) {
        alert("Enter valid salary");
        return false;
      }

      alert("Registration Successful!");
      return true;
    }
  </script>
</head>
<body>
```

Q.1) Create a node.js file that Select all records from the "students" table, and display the result object on console.

__app.js

```
const mysql = require("mysql");

const con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "your_database_name"
});

con.connect(function(err) {
  if (err) throw err;

  con.query("SELECT * FROM students", function(err, result) {
    if (err) throw err;
    console.log(result);
  });
});
```

__Run

```
npm install mysql
node app.js
```

Q.2) Create an HTML form for Employee and write a JavaScript to validate name, email ID, mobile number, department, joining date using Regular Expression.

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Form</title>
  <script>
    function validateForm() {
      let name = document.getElementById("name").value.trim();
      let email = document.getElementById("email").value.trim();
      let mobile = document.getElementById("mobile").value.trim();
      let dept = document.getElementById("dept").value.trim();
      let join = document.getElementById("join").value;

      let namePattern = /^[A-Za-z ]+$/;
      let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      let mobilePattern = /^[0-9]{10}$/;
      let deptPattern = /^[A-Za-z ]+$/;

      if (!namePattern.test(name)) {
        alert("Enter valid name");
        return false;
      }

      if (!emailPattern.test(email)) {
        alert("Enter valid email");
        return false;
      }
    }
  </script>
</head>
<body>
  <div>
    <input type="text" id="name" value="Name" />
    <input type="text" id="email" value="Email" />
    <input type="text" id="mobile" value="Mobile" />
    <input type="text" id="dept" value="Department" />
    <input type="text" id="join" value="Joining Date" />
  </div>
  <div>
    <button type="button" value="Validate" />
  </div>
</body>
</html>
```

```

        if (!mobilePattern.test(mobile)) {
            alert("Enter valid 10-digit mobile number");
            return false;
        }

        if (!deptPattern.test(dept)) {
            alert("Enter valid department");
            return false;
        }

        if (join === "") {
            alert("Select joining date");
            return false;
        }

        alert("Form submitted successfully!");
        return true;
    }
</script>
</head>
<body>

<h2>Employee Registration Form</h2>

<form onsubmit="return validateForm()">

    Name:<br>
    <input type="text" id="name" required><br><br>

    Email ID:<br>
    <input type="text" id="email" required><br><br>

    Mobile No:<br>
    <input type="text" id="mobile" required><br><br>

    Department:<br>
    <input type="text" id="dept" required><br><br>

    Joining Date:<br>
    <input type="date" id="join" required><br><br>

    <input type="submit" value="Submit">

</form>

</body>
</html>

```

SLIP 16

Q.1) Using node js create a Recipe Book.

__(app.js)

```

const express = require("express");
const bodyParser = require("body-parser");

const app = express();

```

```

app.use(bodyParser.urlencoded({ extended: true }));

let recipes = [];

app.get("/", function(req, res) {
  let html = `
    <h2>Recipe Book</h2>
    <form method="post" action="/add">
      Recipe Name:<br>
      <input type="text" name="name"><br><br>
      Ingredients:<br>
      <input type="text" name="ingredients"><br><br>
      <input type="submit" value="Add Recipe">
    </form>
    <h3>Recipes:</h3>
    <ul>
  `;

  recipes.forEach(function(r) {
    html += `<li><b>${r.name}</b>: ${r.ingredients}</li>`;
  });

  html += "</ul>";
  res.send(html);
});

app.post("/add", function(req, res) {
  recipes.push({
    name: req.body.name,
    ingredients: req.body.ingredients
  });
  res.redirect("/");
});

app.listen(3000);

```

__Run
 npm init -y
 npm install express body-parser
 node app.js

Open:
 http://localhost:3000

Q.2) Create a js file named main.js for event-driven application. There should be a main loop that listens for events, and then triggers a call-back function when one of those events is detected.

__main.js

```

const EventEmitter = require("events");

const emitter = new EventEmitter();

emitter.on("start", function() {
  console.log("Start event triggered");
});

emitter.on("process", function(data) {
  console.log("Processing:", data);
});

```



```

emitter.on("end", function() {
    console.log("End event triggered");
});

console.log("Main loop listening for events...");

setTimeout(function() {
    emitter.emit("start");
}, 1000);

setTimeout(function() {
    emitter.emit("process", "Sample Data");
}, 2000);

setTimeout(function() {
    emitter.emit("end");
}, 3000);

__Run
node main.js

```

SLIN 17

Q.1) Using angular js Create a SPA that show Syllabus content of all subjects of M.Sc (CS) Sem-II (use ng-view)

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
    <title>MSc CS Sem-II Syllabus</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-
route.js"></script>
</head>
<body>

<h2>M.Sc (CS) Sem-II Syllabus</h2>

<a href="#!ds">Data Science</a> |
<a href="#!os">Operating System</a> |
<a href="#!java">Java</a>

<div ng-view></div>

<script>
var app = angular.module("myApp", ["ngRoute"]);

app.config(function($routeProvider) {
    $routeProvider
        .when("/ds", {
            template: "<h3>Data Science</h3><p>Data preprocessing, visualization, ML
basics</p>"
        })
        .when("/os", {
            template: "<h3>Operating System</h3><p>Processes, scheduling, memory

```

```

management</p>"
    })
    .when("/java", {
        template: "<h3>Java</h3><p>OOP concepts, collections, exception
handling</p>"
    })
    .otherwise({
        template: "<h3>Welcome</h3><p>Select a subject to view syllabus</p>"
    });
});
</script>

</body>
</html>

```

Q.2) Using angular js display the 10 student details in Table format
(using ng-repeat directive use Array to store data)

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
    <title>Student Table</title>
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>Student Details</h2>

<table border="1" cellpadding="10">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Age</th>
    </tr>

    <tr ng-repeat="s in students">
        <td>{{s.id}}</td>
        <td>{{s.name}}</td>
        <td>{{s.age}}</td>
    </tr>
</table>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope) {
    $scope.students = [
        {id: 1, name: "Amit", age: 20},
        {id: 2, name: "Rahul", age: 21},
        {id: 3, name: "Sneha", age: 19},
        {id: 4, name: "Priya", age: 22},
        {id: 5, name: "Kiran", age: 20},
        {id: 6, name: "Neha", age: 21},
        {id: 7, name: "Rohit", age: 23},
        {id: 8, name: "Pooja", age: 20},
        {id: 9, name: "Vikas", age: 22},
        {id: 10, name: "Anjali", age: 19}
    ];
});
</script>

```

```
</body>
</html>
```

SLIUP 18

Q.1) Using node js create a User Login System

__1. index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>

<h2>User Login</h2>

<form action="/login" method="post">
  Username:<br>
  <input type="text" name="username" required><br><br>

  Password:<br>
  <input type="password" name="password" required><br><br>

  <input type="submit" value="Login">
</form>

</body>
</html>
```

__2. app.js

```
const express = require("express");
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

const user = {
  username: "admin",
  password: "1234"
};

app.get("/", function(req, res) {
  res.sendFile(__dirname + "/index.html");
});

app.post("/login", function(req, res) {
  const { username, password } = req.body;

  if (username === user.username && password === user.password) {
    res.send("<h2>Login Successful</h2>");
  } else {
    res.send("<h2>Invalid Username or Password</h2>");
  }
});
```

```
});  
  
app.listen(3000);  
  
__Run  
npm init -y  
npm install express body-parser  
node app.js  
  
Open:  
http://localhost:3000
```

Q.2) Create a node.js file that Select all records from the "customers" table, and find the customers whose name starts from "A".

```
__app.js  
  
const mysql = require("mysql");  
  
const con = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "",  
  database: "your_database_name"  
});  
  
con.connect(function(err) {  
  if (err) throw err;  
  
  con.query("SELECT * FROM customers", function(err, result) {  
    if (err) throw err;  
    console.log("All Customers:");  
    console.log(result);  
  });  
  
  con.query("SELECT * FROM customers WHERE name LIKE 'A%", function(err,  
result) {  
    if (err) throw err;  
    console.log("Customers whose name starts with A:");  
    console.log(result);  
  });  
});  
  
__Run  
npm install mysql  
node app.js
```

SLIP 19

Q.1) Create a Node.js file that will convert the output "Hello World!" into upper-case letters.

```
__app.js
```

```

let str = "Hello World!";

let upper = str.toUpperCase();

console.log("Original String:", str);
console.log("Uppercase String:", upper);

```

```

__Run
node app.js

```

Q.2) Using angular js create a SPA to accept the details such as name, mobile number, pin code and email address and make validation. Name should contain character only, address should contain SPPU M.Sc. Computer Science Syllabus 2023-24, mobile number should contain only 10 digit, Pin code should contain only 6 digit, email id should contain only one @, . Symbol.

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Student Details SPA</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="myController">

<h2>Student Details Form</h2>

<form ng-submit="validate()">

  Name:<br>
  <input type="text" ng-model="name"><br><br>

  Address:<br>
  <input type="text" ng-model="address"><br><br>

  Mobile:<br>
  <input type="text" ng-model="mobile"><br><br>

  Pin Code:<br>
  <input type="text" ng-model="pincode"><br><br>

  Email:<br>
  <input type="text" ng-model="email"><br><br>

  <input type="submit" value="Submit">
</form>

<h3>{{message}}</h3>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope) {

  $scope.validate = function() {

    let namePattern = /^[A-Za-z ]+$/;
    let mobilePattern = /^[0-9]{10}$/;

```

```

        let pinPattern = /^[0-9]{6}$/;
        let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        if (!namePattern.test($scope.name)) {
            alert("Name should contain only characters");
            return;
        }

        if ($scope.address !== "SPPU M.Sc. Computer Science Syllabus 2023-24") {
            alert("Address must be exactly: SPPU M.Sc. Computer Science Syllabus
2023-24");
            return;
        }

        if (!mobilePattern.test($scope.mobile)) {
            alert("Mobile number must be 10 digits");
            return;
        }

        if (!pinPattern.test($scope.pincodes)) {
            alert("Pin code must be 6 digits");
            return;
        }

        if (!emailPattern.test($scope.email)) {
            alert("Enter valid email");
            return;
        }

        $scope.message = "All details are valid!";
    };
});
</script>

</body>
</html>

```

SLIP 20

Q.1) Create a Node.js file that demonstrate create student database and table in MySQL

__app.js

```

const mysql = require("mysql");

const con = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: ""
});

con.connect(function(err) {
    if (err) throw err;

    con.query("CREATE DATABASE studentdb", function(err, result) {
        if (err) throw err;
        console.log("Database created");
    });
});

```

```

});

const con2 = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "studentdb"
});

con2.connect(function(err) {
  if (err) throw err;

  const sql = "CREATE TABLE students (id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(50), age INT)";
  con2.query(sql, function(err, result) {
    if (err) throw err;
    console.log("Table created");
  });
});
});
});

```

__Run
 npm install mysql
 node app.js

Q.2) Using angular js create a SPA to carry out validation for a username entered in a textbox.

If the textbox is blank, alert "Enter username" . If the number of characters is less than three, alert "Username is too short" . If value entered is appropriate the print "Valid username" and password should be minimum 8 characters

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Username Validation</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>User Validation Form</h2>

<form ng-submit="validate()">

  Username:<br>
  <input type="text" ng-model="username"><br><br>

  Password:<br>
  <input type="password" ng-model="password"><br><br>

  <input type="submit" value="Submit">
</form>

<h3>{{message}}</h3>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope) {

```

```

$scope.validate = function() {
    if (!$scope.username || $scope.username.trim() === "") {
        alert("Enter username");
        return;
    }

    if ($scope.username.length < 3) {
        alert("Username is too short");
        return;
    }

    if (!$scope.password || $scope.password.length < 8) {
        alert("Password should be minimum 8 characters");
        return;
    }

    $scope.message = "Valid username";
};
});
</script>

</body>
</html>

```

SLIP 21

Q.1) Create a Node.js file that demonstrate create Movie database and table in MySQL

__app.js

```

const mysql = require("mysql");

const con = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: ""
});

con.connect(function(err) {
    if (err) throw err;

    con.query("CREATE DATABASE movie_db", function(err, result) {
        if (err) throw err;
        console.log("Database created");
    });

    const con2 = mysql.createConnection({
        host: "localhost",
        user: "root",
        password: "",
        database: "movie_db"
    });

```



```

    con2.connect(function(err) {
        if (err) throw err;

        const sql = "CREATE TABLE movies (id INT AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(100), year INT)";
        con2.query(sql, function(err, result) {
            if (err) throw err;
            console.log("Table created");
        });
    });
});

__Run
npm install mysql
node app.js

```

Q.2) Write node js application that transfer a file as an attachment on web and enables browser to prompt the user to download file using express js.

```

__app.js

const express = require("express");
const path = require("path");

const app = express();

app.get("/download", function(req, res) {
    const filePath = path.join(__dirname, "file.txt");
    res.download(filePath);
});

app.listen(3000);

__Run
npm install express
node app.js

Open:
http://localhost:3000/download

```

SLIP 22

Q.1) Using node js create an Employee Registration Form validation.

```

__1. index.html

<!DOCTYPE html>
<html>
<head>
    <title>Employee Registration</title>
</head>
<body>

<h2>Employee Registration Form</h2>

<form action="/register" method="post">

```

```

Name:<br>
<input type="text" name="name"><br><br>

Email:<br>
<input type="text" name="email"><br><br>

Mobile:<br>
<input type="text" name="mobile"><br><br>

Salary:<br>
<input type="text" name="salary"><br><br>

<input type="submit" value="Register">
</form>

</body>
</html>

```

__2. app.js

```

const express = require("express");
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});

app.post("/register", (req, res) => {
  const { name, email, mobile, salary } = req.body;

  const namePattern = /^[A-Za-z ]+$/;
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const mobilePattern = /^[0-9]{10}$/;
  const salaryPattern = /^[0-9]+$/;

  if (!namePattern.test(name)) {
    return res.send("Invalid Name");
  }

  if (!emailPattern.test(email)) {
    return res.send("Invalid Email");
  }

  if (!mobilePattern.test(mobile)) {
    return res.send("Invalid Mobile Number");
  }

  if (!salaryPattern.test(salary)) {
    return res.send("Invalid Salary");
  }

  res.send("Employee Registered Successfully");
});

app.listen(3000);

```

__Run
 npm init -y

```
npm install express body-parser
node app.js
```

Open:
<http://localhost:3000>

Q.2) Create a js file named main.js for event-driven application. There should be a main loop that listens for events, and then triggers a callback function when one of those events is detected.

__main.js

```
const EventEmitter = require("events");

const eventEmitter = new EventEmitter();

eventEmitter.on("login", function(user) {
  console.log("User logged in:", user);
});

eventEmitter.on("logout", function(user) {
  console.log("User logged out:", user);
});

console.log("Main loop listening for events...");

setTimeout(function() {
  eventEmitter.emit("login", "Admin");
}, 2000);

setTimeout(function() {
  eventEmitter.emit("logout", "Admin");
}, 4000);
```

__Run
node main.js

SLIP 23

Q.1) Write node js script to interact with the file system, and serve a web page from a File

__app.js

```
const http = require("http");
const fs = require("fs");

http.createServer(function(req, res) {
  fs.readFile("index.html", function(err, data) {
    if (err) {
      res.writeHead(404, { "Content-Type": "text/html" });
    }
  });
});
```

```

        res.end("404 Not Found");
    } else {
        res.writeHead(200, { "Content-Type": "text/html" });
        res.write(data);
        res.end();
    }
});
}).listen(3000);

```

____Run
node app.js

Open:
http://localhost:3000

__index.html

```

<!DOCTYPE html>
<html>
<head>
    <title>File System Web Page</title>
</head>
<body>

<h2>Welcome to Node.js File System Example</h2>
<p>This page is served from a file using fs module.</p>

</body>
</html>

```

Q.2) Write node js script to build Your Own Node.js Module. Use require (âhttpâ) module is a built-in Node module that invokes the functionality of the HTTP library to create a local server. Also use the export statement to make functions in your module available externally. Create a new text file to contain the functions in your module called, âmodules.jsâ and add this function to return todayâs date and time.

____1. modules.js (Custom Module)

```

exports.getDateTime = function() {
    return new Date();
};

```

____2. main.js

```

const http = require("http");
const myModule = require("./modules");

http.createServer(function(req, res) {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.write("Current Date and Time: " + myModule.getDateTime());
    res.end();
}).listen(3000);

```

____Run
node main.js

Open:
http://localhost:3000

SLIP 24

Q.1) Using node js create an eLearning System

```
<!DOCTYPE html>
<html>
<head>
  <title>eLearning System</title>
</head>
<body>

<h2>Available Courses</h2>

<form action="/enroll" method="post">
  <label>Name:</label><br>
  <input type="text" name="name" required><br><br>

  <label>Select Course:</label><br>
  <select name="course">
    <option value="Full Stack Development">Full Stack Development</option>
    <option value="Data Science">Data Science</option>
    <option value="Machine Learning">Machine Learning</option>
  </select><br><br>

  <input type="submit" value="Enroll">
</form>

</body>
</html>
```

__ Node.js File (app.js)

```
const express = require("express");
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});

app.post("/enroll", (req, res) => {
  const name = req.body.name;
  const course = req.body.course;

  res.send("<h2>" + name + " enrolled in " + course + " successfully!</h2>");
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

__Steps to Run

```
npm init -y
npm install express body-parser
node app.js
```

Q.2) Using angular js create a SPA to carry out validation for a username entered in a textbox.

If the textbox is blank, alert "Enter username" . If the number of characters is less than three, alert "Username is too short" . If value entered is appropriate the print "Valid username" and password should be minimum 8 characters

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Username Validation SPA</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>User Validation</h2>

<form ng-submit="validate()">

  Username:<br>
  <input type="text" ng-model="username"><br><br>

  Password:<br>
  <input type="password" ng-model="password"><br><br>

  <input type="submit" value="Submit">
</form>

<h3>{{message}}</h3>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope) {

  $scope.validate = function() {

    if (!$scope.username || $scope.username.trim() === "") {
      alert("Enter username");
      return;
    }

    if ($scope.username.length < 3) {
      alert("Username is too short");
      return;
    }

    if (!$scope.password || $scope.password.length < 8) {
      alert("Password should be minimum 8 characters");
      return;
    }

    $scope.message = "Valid username";
  };
};
```

```
});
</script>

</body>
</html>
```

SLIP 25

Q.1) Create an angular JS Application that shows the location of the current web page.

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Page Location</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
</head>
<body ng-controller="myController">

<h2>Current Web Page Location</h2>

<p><b>URL:</b> {{url}}</p>
<p><b>Protocol:</b> {{protocol}}</p>
<p><b>Host:</b> {{host}}</p>
<p><b>Path:</b> {{path}}</p>

<script>
var app = angular.module("myApp", []);

app.controller("myController", function($scope, $location) {

  $scope.url = $location.absUrl();
  $scope.protocol = $location.protocol();
  $scope.host = $location.host();
  $scope.path = $location.path();

});
</script>

</body>
</html>
```

Q.2) Create a js file named main.js for event-driven application. There should be a main loop that listens for events, and then triggers a callback function when one of those events is detected.

___main.js

```
const EventEmitter = require("events");

const eventEmitter = new EventEmitter();
```

```
eventEmitter.on("start", function() {
  console.log("Start event detected");
});

eventEmitter.on("process", function(data) {
  console.log("Processing data:", data);
});

eventEmitter.on("end", function() {
  console.log("End event detected");
});

console.log("Main loop is listening for events...");

setTimeout(function() {
  eventEmitter.emit("start");
}, 1000);

setTimeout(function() {
  eventEmitter.emit("process", "Sample Input");
}, 2000);

setTimeout(function() {
  eventEmitter.emit("end");
}, 3000);
```

___Run
node main.js

@#\$\$%^&*